

VEHICLE MAINTENANCE TRACKER

FULL-STACK MERN APPLICATION — INTERVIEW & VIVA PRESENTATION GUIDE

Complete Technical Blueprint, Architecture Deep-Dive & 28 Model Interview Q&As

1. Executive Summary & Project Pitch

Vehicle Maintenance Tracker is a full-stack MERN (MongoDB, Express.js, React, Node.js) web application designed for personal vehicle owners and fleet managers. It streamlines vehicle record-keeping, service log tracking, spending analytics visualization, and automated reminder alerts for upcoming or overdue vehicle maintenance tasks.

The 30-Second Elevator Pitch (For Interviewers)

"I built a full-stack Vehicle Maintenance Tracker using the MERN stack. It allows users to manage vehicle profiles, log service histories, and visualize maintenance costs through interactive charts. What sets it apart is its automated background reminder system built with node-cron and Nodemailer, which monitors service due dates independently of user requests and dispatches in-app notifications and email alerts. It also uses MongoDB aggregation pipelines to serve real-time financial analytics."

Key Technical Differentiators

- **Automated Background Processing:** Integrates a daily node-cron background job that scans MongoDB for due/overdue service records and creates in-app + email notifications automatically.
- **Advanced MongoDB Aggregation:** Uses \$group, \$lookup, \$unwind, and date arithmetic operators to compute spend totals, monthly trends, and expense breakdowns.
- **Custom Visualization Components:** Features a signature radial Speedometer Gauge (SVG path arc calculation) showing days until next service alongside Recharts financial analytics.
- **Production Security:** Stateless JWT auth with bearer token interceptors, salted password hashing via bcryptjs, and strict user-scoped data authorization.

2. System Architecture & Tech Stack Rationale

The application follows a decoupled Client-Server architecture with a RESTful API contract and a background job scheduler.

- **Frontend Stack:** React 18 (Vite build tool), Material-UI (MUI v5) for responsive layout and dark theme design system, Recharts for financial visual analytics, Axios for HTTP client with JWT request interceptors, React Router v6 for client-side routing.
- **Backend Stack:** Node.js runtime, Express.js framework for REST API routing and centralized error handling middleware, Mongoose ODM for MongoDB object modeling, bcryptjs for password encryption, jsonwebtoken for JWT issuance/verification.
- **Background & Services:** node-cron for scheduled daily reminder jobs at 08:00 AM, Nodemailer for SMTP email notifications, MongoDB Atlas cloud cluster for scalable JSON document persistence.

3. Database Schemas & Data Modeling

- **User Model:** fields: `_id`, `name` (String), `email` (String, unique), `password` (String, hashed, select: false), `role` ("user" | "admin"). Methods: `matchPassword()` for bcrypt comparison.
- **Vehicle Model:** fields: `_id`, `owner` (Ref: User), `nickname` (String), `make` (String), `model` (String), `year` (Number), `registrationNumber` (String, uppercase), `fuelType` (Enum), `currentOdometer` (Number), `imageColor` (Hex string).
- **ServiceRecord Model:** fields: `_id`, `vehicle` (Ref: Vehicle), `owner` (Ref: User), `serviceType` (String), `description` (String), `cost` (Number), `odometerAtService` (Number), `serviceDate` (Date), `nextDueDate` (Date), `nextDueOdometer` (Number), `status` ("completed" | "upcoming" | "overdue"), `reminderSent` (Boolean).
- **Notification Model:** fields: `_id`, `owner` (Ref: User), `vehicle` (Ref: Vehicle), `serviceRecord` (Ref: ServiceRecord), `title` (String), `message` (String), `type` ("reminder" | "overdue" | "info"), `isRead` (Boolean).

4. Complete REST API Specifications

METHOD	ENDPOINT	ACCESS	DESCRIPTION
POST	/api/auth/register	Public	Create user account & return JWT
POST	/api/auth/login	Public	Authenticate user & return JWT
GET	/api/auth/me	Private	Get currently logged-in user profile
GET / POST	/api/vehicles	Private	Fetch user vehicles / Create new vehicle
GET/PUT/DEL	/api/vehicles/:id	Private	Read, update, or delete vehicle + records
GET / POST	/api/services	Private	List service records (?vehicle=id) / Log record
GET/PUT/DEL	/api/services/:id	Private	Read, update, or delete service record
GET	/api/analytics/summary	Private	Get total spend, vehicle count, upcoming/overdue
GET	/api/analytics/monthly	Private	Monthly spend series (?months=6) for line chart
GET	/api/analytics/by-type	Private	Spend grouped by service type for pie chart
GET	/api/analytics/by-vehicle	Private	Spend grouped by vehicle for bar chart
GET	/api/notifications	Private	Fetch notifications for user
PUT	/api/notifications/read-all	Private	Mark all user notifications as read
POST	/api/dev/run-reminder-check	Private	Manual trigger for reminder check job

5. Comprehensive Viva & Interview Questions & Answers

Q1: What is the core purpose of this application and who is the target user?

The application helps vehicle owners and fleet managers track maintenance costs, maintain complete service histories, visualize spending trends, and receive automated reminders before maintenance becomes overdue. It solves the common problem of missed oil changes, forgotten service schedules, and untracked vehicle expenditure.

Q2: Can you explain the high-level architecture of your application?

It follows a decoupled MERN stack architecture. The React frontend handles user interactions and charts, communicating asynchronously via Axios REST requests. The Express backend handles routing, JWT authentication middleware, and business logic. Mongoose interfaces with MongoDB Atlas for persistent storage. In parallel, a node-cron background process runs on the server to execute daily reminder scans.

Q3: What are the key technical highlights of this project?

1. Automated node-cron + Nodemailer background notifications.
2. Complex MongoDB aggregation pipelines (\$group, \$lookup, \$unwind) for financial analytics.
3. Dynamic SVG Radial Speedometer Gauge component built from mathematical polar-to-cartesian conversions.
4. Robust JWT auth with custom Express error-handling middleware.

Q4: Why did you choose Vite over Create React App (CRA)?

Vite uses native ES modules during development and leverage Esbuild (written in Go) for pre-bundling, making dev server startup and Hot Module Replacement (HMR) virtually instantaneous compared to CRA's Webpack setup. For production, Vite uses Rollup for highly optimized chunking.

Q5: Why did you choose MongoDB over a SQL database like PostgreSQL?

Vehicle service records and vehicle attributes can vary (e.g. electric vehicles don't need oil changes, fuel types vary, arbitrary notes). Document-oriented MongoDB offers schema flexibility while Mongoose allows strict schema enforcement where required. MongoDB aggregations also make time-series spending summaries straightforward.

Q6: How is CORS handled between the frontend and backend?

The backend uses the cors middleware in Express configured with origin: `process.env.CLIENT_URL || "http://localhost:5173"`. This allows browser pre-flight OPTIONS requests from the React client while blocking unauthorized cross-origin origins in production.

Q7: Walk me through your JWT authentication flow.

1. User submits login form with email & password.
2. Backend authController looks up user, fetching password (+select("+password")).
3. bcrypt.compare checks password match.
4. Backend signs a JWT with user._id payload and secret key, returning token + user profile.
5. Frontend stores token in localStorage and sets Axios Authorization header (Bearer <token>).
6. Protected routes run auth middleware which verifies token via jwt.verify, attaches req.user, and calls next().

Q8: How are passwords secured in MongoDB?

Passwords are never stored in plain text. A Mongoose pre("save") hook uses bcryptjs to generate a salt (factor 10) and hash the password before saving. The field is configured with select: false in the schema so database queries exclude it by default unless explicitly requested during authentication.

Q9: What happens when a JWT token expires?

The auth middleware throws a TokenExpiredError. The backend returns a 401 Unauthorized status. On the frontend, an Axios response interceptor catches 401 responses, clears localStorage (vmt_token, vmt_user), and redirects the user to / login.

Q10: Explain the MongoDB aggregation pipeline for monthly expenses.

It uses a \$match stage to filter records for req.user._id within a date range (e.g., last 6 months). Then a \$group stage groups by year and month ({ year: { \$year: "\$serviceDate" }, month: { \$month: "\$serviceDate" } }), computing \$sum of cost. In Node.js, we loop through missing months to zero-fill data points so line charts display continuous trends without missing months.

Q11: How does the expense by vehicle endpoint connect ServiceRecord with Vehicle?

It uses a \$lookup aggregation stage: { \$lookup: { from: "vehicles", localField: "_id", foreignField: "_id", as: "vehicle" } } followed by \$unwind: "\$vehicle" to flatten the array, allowing the backend to aggregate costs by vehicle nickname or make/model.

Q12: What happens to service records when a vehicle is deleted?

In vehicleController.deleteVehicle, we execute ServiceRecord.deleteMany({ vehicle: vehicle._id }) before removing the vehicle document itself. This prevents orphaned service records in the database.

Q13: How does the node-cron background reminder job operate?

When the server starts, scheduleReminderJob() registers a cron schedule ("0 8 * * *", daily at 8 AM). When triggered, runReminderCheck() queries service records where nextDueDate is within REMINDER_DAYS_BEFORE (3 days) and reminderSent is false. It creates a Notification document and sends an email via Nodemailer.

Q14: What happens if email sending fails (e.g. SMTP credentials invalid or down)?

The sendEmail utility wraps transporter.sendMail in a try-catch block. If unconfigured or throwing an error, it logs a warning/error without crashing the cron job or preventing the in-app Notification record from being created.

Q15: How can an interviewer or tester test reminders without waiting until 8:00 AM?

We created a developer test route POST /api/dev/run-reminder-check. Calling this endpoint manually triggers runReminderCheck() immediately, evaluating due dates and populating notifications instantly.

Q16: How does the signature Speedometer Gauge (ServiceGauge) work?

It is a custom React SVG component. It calculates an arc path using trigonometry (polarToCartesian) mapping days remaining (0 to 90 days) to needle angles (-120deg to +120deg). The color dynamically shifts from Teal (healthy, >14d) to Amber (upcoming, 0-14d) to Rust-Red (overdue, <0d).

Q17: How did you build the custom dark theme in Material-UI?

Using MUI createTheme with palette mode "dark". Custom background colors (#1C2128 charcoal base, #2D333B card graphite), custom typography (Space Grotesk for headings, JetBrains Mono for numbers/monetary values, Inter for body), and overridden component styles for MuiCard, MuiButton, and MuiAppBar.

Q18: How is state shared across components?

User authentication state (user, token, login, logout) is managed globally via React Context API (AuthContext). Resource data (vehicles, services, notifications) is fetched per page/component and refreshed upon create/update/delete operations.

Q19: How do you ensure odometer readings remain accurate across service logs?

When a user creates a new service record with odometerAtService, serviceController checks if odometerAtService > vehicle.currentOdometer. If higher, it automatically updates the vehicle document's currentOdometer field.

Q20: How would you scale node-cron if this backend ran on multiple server instances (e.g., behind a Load Balancer)?

If multiple Node instances run node-cron, duplicate emails/notifications would fire. To scale, I would decouple the cron runner into a single dedicated worker process, or use Redis-backed queue/scheduler like BullMQ with distributed locks (Redlock).

Q21: How would you handle database indexing for high query volume?

We added a compound index on ServiceRecord: { owner: 1, serviceDate: -1 }. This optimizes queries filtering by user ID and sorting chronologically for dashboard and analytics pipelines.

Q22: How is input validation performed?

Backend Mongoose schemas enforce required fields, string trimming, email regex patterns, min length for passwords, and number ranges. Controllers check required fields before processing and throw descriptive errors handled by centralized errorHandler middleware.

Q23: What is the role of centralized error-handling middleware in Express?

In Express, middleware with 4 arguments (err, req, res, next) acts as a catch-all. It catches Mongoose validation errors, duplicate key errors (code 11000), and CastErrors (invalid ObjectIds), mapping them to HTTP status codes (400, 404, 500) and returning clean JSON responses.

Q24: How do protected routes prevent unauthorized users from viewing pages on the frontend?

The ProtectedRoute component wraps private pages. It checks user/token status from AuthContext. If no user/token exists, it renders <Navigate to="/login" replace />.

Q25: How are status badges dynamically calculated for service records?

computeStatus(nextDueDate) compares nextDueDate to Date.now(). If past due date -> "overdue". If within 14 days -> "upcoming". Otherwise -> "completed". Status is recomputed on creation, update, and during daily cron scans.

Q26: What libraries were used for charts and why?

Recharts was chosen because it is built natively for React using SVG elements, responsive via <ResponsiveContainer>, seamlessly integrates with custom theme colors, and has clean declarative API (<LineChart>, <BarChart>, <PieChart>).

Q27: If given 2 more weeks, what features would you add next?

1. File upload capability (Multer + AWS S3) for service receipt images & PDF invoices.
2. Export to CSV/PDF feature for tax/expense reporting.
3. Maintenance cost predictions based on vehicle odometer mileage trajectories.

Q28: What was the most challenging part of developing this application?

Ensuring continuous date series in MongoDB aggregations when months had zero spend, and cleanly handling asynchronous cron execution without blocking the main Express HTTP server event loop.

6. Live Interview Demo Script & Presentation Tips

Recommended 5-Minute Live Demo Walkthrough

- **Step 1: Introduction (30 sec):** Open the browser to <http://localhost:5173/login>. Briefly state the project goal: "Full-stack vehicle maintenance & automated expense tracker."
- **Step 2: Authentication (45 sec):** Log in with alex@example.com / password123. Highlight JWT storage in localStorage and automatic HTTP header attachment in Axios.
- **Step 3: Garage Dashboard (1 min):** Point out the Stat Cards (Total Spend, Vehicle count, Upcoming/Overdue counts) and the custom Speedometer Gauge for vehicle service readiness.
- **Step 4: Vehicle & Service Log CRUD (1.5 min):** Navigate to Vehicles & Services. Click "Log Service" to create a service record. Show how vehicle odometer updates automatically.
- **Step 5: Analytics Charts (1 min):** Navigate to Analytics. Show the Recharts Line chart (monthly spend), Pie chart (service types), and Bar chart (spend by vehicle). Explain the backend aggregation pipelines.
- **Step 6: Automated Reminders (45 sec):** Click the Notification bell icon in the Navbar to show unread reminder alerts. Explain how node-cron & Nodemailer execute daily background checks.

Golden Interview Presentation Tips

- **Speak in Terms of Data Flow:** Always trace actions end-to-end: UI Event -> State -> API Request -> Express Middleware -> Controller -> MongoDB -> Response -> Re-render.
- **Highlight Edge Cases Handled:** Mention zero-spend month filling, auto-odometer sync, non-crashing email fallbacks, and JWT expiration interceptors.
- **Be Confident on MongoDB Aggregations:** Interviewers love asking about backend analytics. Be ready to explain \$group and \$lookup on the whiteboard.

